

Unity 玩家与近战敌人系统拆解案

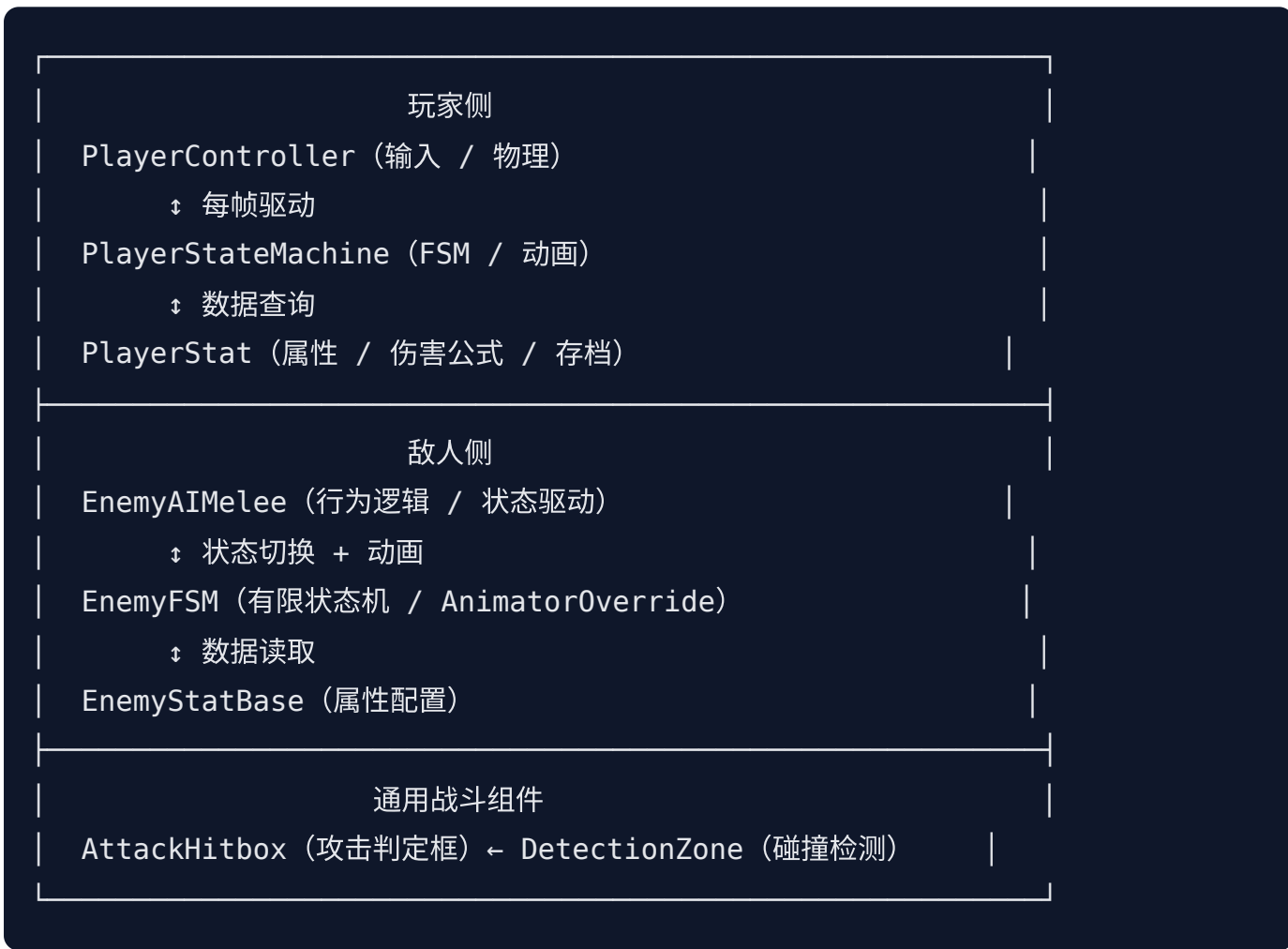
作者：费锦意

该系统模块应用于本人原创游戏《晦境孤剑：残玉为王》

Unity 2D · C# · 有限状态机 · 事件驱动架构

一、整体架构概览

系统分为**玩家侧**与**敌人侧**两条并行架构，共用 `DetectionZone` / `AttackHitbox` 通用组件完成双向战斗交互。



核心设计理念：

- **MVC 分层**：输入/物理（Controller）、状态/动画（StateMachine/FSM）、数据（Stat）三层严格分离
- **有限状态机（FSM）驱动**：玩家与敌人均以 FSM 管理所有行为状态，避免 if-else 嵌套膨胀
- **事件回调解耦战斗**：AttackHitbox.OnHit 事件将攻击判定与伤害逻辑完全解耦
- **通用检测组件复用**：DetectionZone 同时服务于视野检测、攻击启动、转身检测、受击判定

二、玩家侧

2.1 PlayerController（输入层 / 物理层）

类型：MonoBehaviour，挂载于玩家根节点

PlayerController 是玩家系统的最外层，专职负责**原始输入采集、物理运动执行、地面检测**，不包含任何状态判断逻辑。

输入缓存机制

```
Update() 每帧
    → 读取 Input.GetKeyDown / GetKey → 写入布尔缓存 (jumpInput / attackInput 等)
    → HandlePlayerInput() : 消费缓存, 调用 StateMachine 对应接口, 消费后立即置 false
```

将输入读取与状态响应分为两步，确保在同一帧内输入不会被重复消费，且状态机可在任意时机安全调用。

地面检测

```
isGrounded = Physics2D.Raycast(groundCheck.position, Vector2.down,
    groundCheckDistance, groundLayer);
```

使用射线检测代替 OnCollisionStay，精度更高，不受碰撞体边缘误差影响。groundCheck 子节点位置与 groundCheckDistance 均可在 Inspector 可视化调节，OnDrawGizmosSelected 提供编辑器可视化辅助。

攻击突进系统

```
// 每次攻击进入时设置剩余距离
public void StartAttackMove(float distance) → attackMoveRemaining = distance

// FixedUpdate 中逐帧消耗
public void UpdateAttackMove()
    → moveThisFrame = attackMoveRemaining * 0.3f (指数衰减, 起步猛收尾缓)
    → rb.MovePosition(...)
    → attackMoveRemaining 归零后锁定水平速度为 0
```

三段攻击分别配置不同的 `attackMoveDistance` (1f / 1.5f / 2f), 实现递进式突进手感。

翻转锁定

攻击期间由状态机将 `canFlip = false`, 并通过 `flipBuffered` 标记在动画结束后延迟执行翻转, 避免攻击中途因方向键变化导致模型乱转。

2.2 PlayerStateMachine (FSM 核心层)

类型: `MonoBehaviour`, 由 `PlayerController` 每帧主动调用 `CustomUpdate()` / `CustomFixedUpdate()`

状态枚举

```
Idle | Run | Jump | Fall | ToGround | Slide
Attack1 | Attack2 | Attack3 | HeavyAttack
Hurt | Die | UseBlessing | Getup
```

共 14 个状态, 覆盖移动、战斗、受击、死亡完整生命周期。

驱动模型

```

PlayerController.Update()
└─ stateMachine.CustomUpdate()
    │─ stateTimer / hurtCooldown / slideCooldown 计时
    │─ HandleStateLogic() ← 每帧状态内部逻辑
    │─ CheckAutoTransitions() ← 全局自动跌落检测
    └─ HandleSpecialAnimations() ← 跳跃帧冻结处理

PlayerController.FixedUpdate()
└─ stateMachine.CustomFixedUpdate() ← 物理速度设置

```

不依赖 Unity 自带的 `Update`，而由 `PlayerController` 主动驱动，保证执行顺序可控。

状态切换：EnterState / ExitState

```

ChangeState(newState)
→ ExitState(currentState)：清理攻击框、恢复翻转锁定、处理缓冲翻转
→ currentState = newState
→ EnterState(newState)：重置计时器、播放动画、执行进入副作用

```

进入/退出逻辑集中管理，避免状态切换时的遗漏清理。

连击缓冲机制

```

TryAttack() 在攻击状态中被调用
→ 不立即切换，仅将 comboBuffered = true

HandleStateLogic() 检测到 normalizedTime >= 1f (动画完全播完)
→ 若 comboBuffered && CanCombo()
    → 切换到下一段攻击 (Attack1 → Attack2 → Attack3 → Attack1 循环)
→ 否则退出到 Idle / Run

```

严格遵循"必须等当前动画播完才能连击"设计，防止攻击帧提前截断。

攻击框激活时机

```

normalizedTime >= 0.5f (动画播到一半) → attackHitbox.Activate()
ExitState 时 → attackHitbox.Deactivate()

```

攻击判定框在动画前半段不激活，符合动作游戏"蓄力-释放"节奏。

重击蓄力机制

```
StartHeavyCharge() → 进入 HeavyAttack 状态
    → isChargingHeavy = true, animator.speed = 0.5f (慢放蓄力)
    → stateTimer = 0.2f (最短蓄力时间)

ReleaseHeavyAttack()
    → stateTimer > 0 (未蓄满) → 取消, 回退到 previousState
    → stateTimer <= 0 (蓄满) → 正常释放, 触发攻击框, 启动冷却
```

蓄力期间动画速度降低增强表现力，松键时机决定是否真正释放。

跳跃帧冻结

```
HandleSpecialAnimations()
    → Jump 状态: 动画播完但 velocity.y > 0 (仍在上升)
        → animator.speed = 0f, 冻结在跳跃顶帧
    → velocity.y <= 0 → animator.speed = 1f, 自然过渡到 Fall
```

解决跳跃动画比物理上升短时出现的动画提前结束问题。

受击与死亡

```
TakeHurt(damage) (外部敌人调用)
    → stat.TakeDamage(damage) ← 数据层扣血
    → HP <= 0 → ChangeState(Die)
    → HP > 0 && hurtCooldown <= 0 → ChangeState(Hurt), 设置 1 秒受伤冷却
```

受击冷却防止连续受击导致 Hurt 状态不停循环，同时保证死亡判定优先于受击。

2.3 PlayerStat (数据层 / 属性管理)

类型: MonoBehaviour，挂载于玩家根节点

PlayerStat 是纯粹的数据层，不处理任何状态转换，只关心数值的计算与存取。

属性分级系统

属性类别	来源	说明
固定属性	脚本序列化字段	Base_PlayerSpeed / Base_JumpForce / Base_CritRate 等
等级属性	硬编码等级表数组	MaxHPTable / BaseAP1~3Table / BaseHeavyAPTable
祝福加成	BlessingType 枚 举	在 GetAttackDamage 中动态叠加
能力解锁	布尔等级表	CanComboTable / CanHeavyTable / CanJumpIITable

伤害计算公式

```
GetAttackDamage(comboStage, out isCrit)

1. baseAP = GetBaseAttackPower(comboStage) ← 查等级表
2. apMultiplier 根据 currentBlessing 修正 (+10% ~ +200%)
3. currAP = Round(baseAP × apMultiplier)
4. critRate / critMulti 根据祝福二次修正
5. isCrit = Random.value < critRate
6. finalDamage = isCrit ? Round(currAP × critMulti) : currAP
7. LifeSteal_Blessing : 命中后恢复 Curr_HP × 10%
```

祝福系统 (BlessingType)

祝福	AP修正	暴击修正	特殊效果
AP_Blessing	+10%	—	—
LifeSteal_Blessing	-10%	—	命中回血 10%
ThreeHit_Blessing	+10%	—	主动技能 (占位)
LowHurt_Blessing	+10%	—	主动技能 (占位)
SuperHit_Blessing	+200%	率+50% 倍+100%	主动技能 (占位)

Invincible_Blessing	+10%	—	主动技能（占位）
Lightning_Blessing	+40%	率=0	主动技能（占位）
Knockback_Blessing	-10%	—	主动技能（占位）

持久化存档

```
Awake() → LoadData()  
→ File.Exists(savePath) → JsonUtility.FromJson<PlayerData>  
→ 解析 Current_Blessing 字符串 → Enum.Parse → currentBlessing  
  
SaveData()  
→ playerData.Current_Blessing = currentBlessing.ToString()  
→ JsonUtility.ToJson → File.WriteAllText
```

`PlayerData` 为内部可序列化类，存储等级、经验、金币、祝福等全部持久化字段，自包含运行，无需外部 `GameManager`。

受伤优先级

```
TakeDamage(damage)  
→ 优先扣除 Curr_ExtraHealth（额外生命，如天使触发）  
→ 剩余伤害再扣 Curr_HP  
→ Curr_HP = Max(0, Curr_HP - damage)
```

三、通用战斗组件

3.1 DetectionZone（通用碰撞检测区域）

类型： `MonoBehaviour`，挂载于带 `Collider2D` (`isTrigger`) 的子对象

```

public string targetTag;           // 要检测的目标 Tag
public bool TargetInZone;         // 目标是否在区域内（每帧可查）
public Collider2D TargetCollider;
public Vector2 LastKnownPosition;
public event Action<Collider2D> OnTargetEnter;
public event Action<Collider2D> OnTargetExit;

```

通过 `OnTriggerEnter2D` / `Stay2D` / `Exit2D` 维护状态，既支持**事件订阅**（一次性反应），也支持**轮询**（`TargetInZone` 每帧查询），两种模式兼顾。

复用场景：

挂载位置	targetTag	用途
敌人目视检测器	"Player"	发现玩家 → Chase
敌人攻击启动框	"Player"	进入攻击范围 → Attack
敌人转身检测器	"Player"	玩家绕后 → 延迟转身
玩家/敌人受击框	"PlayerHurtBox" / "EnemyHurtBox"	配合 AttackHitbox

3.2 AttackHitbox（攻击判定框）

类型： `MonoBehaviour`，挂载于攻击检测子对象，依赖 `DetectionZone`

```

Activate() → SetActive(true), _hasHit = false（重置单次命中标志）
Deactivate() → SetActive(false)

OnEnable() → 订阅 DetectionZone.OnTargetEnter
OnDisable() → 取消订阅

HandleTargetEnter(target)
    → _hasHit = true（防止一次启用多次命中）
    → OnHit?.Invoke(target)（触发外部伤害回调）
    → gameObject.SetActive(false)（自动禁用）

```

Tag 自动配置：


```
AutoSetTargetTag()  
→ Tag == "PlayerAttackBox" → targetTag = "EnemyHurtBox"  
→ Tag == "EnemyAttackBox" → targetTag = "PlayerHurtBox"
```

挂载时只需设置正确的 GameObject Tag，目标检测类型自动匹配。

兜底逻辑： OnTriggerStay2D 处理攻击框启用时已与目标重叠的情况（Enter 不会重复触发），确保零遗漏。

四、敌人侧

4.1 EnemyStatBase（属性数据层）

类型： MonoBehaviour ，纯数据配置组件

字段	说明
MaxHP / CurHP	最大 / 当前生命值， Awake 中自动初始化
PatrolSpeed / ChaseSpeed	巡逻 / 追逐速度
AttackPower / CriticalRate	攻击力 / 暴击率（百分比）
AttackDelay	攻击冷却时间
IdleToPatrolTimeRange / PatrolToIdleTimeRange	行为切换随机时间范围
KnockbackDistance	受击后被击退距离
FacingRight	朝向标志，供 AI 控制器读取

设计为 MonoBehaviour 便于 Inspector 调节， CurHP 用 [HideInInspector] 隐藏避免混淆配置值与运行时值。子类可 override Awake 扩展初始化逻辑。

4.2 EnemyFSM（敌人有限状态机 / 动画层）

类型： MonoBehaviour ，依赖 Animator

状态枚举

Idle | Patrol | Chase | Attack | AttackCooldown | Hurt | Die

AttackCooldown 为纯逻辑状态，播放 Idle 动画，不单独配置动画资产。

AnimatorOverrideController 懒加载

```
EnsureClipsApplied() (首次播放时触发)
→ new AnimatorOverrideController(runtimeAnimatorController)
→ GetOverrides() → 遍历，按名称匹配替换 Clip
→ ApplyOverrides()
```

基础 AnimatorController 中只需有占位 Clip，运行时自动用 Inspector 赋值的实际 Clip 覆盖，不同敌人共用同一 AnimatorController，各自配置不同 Clip，实现动画资源的高度复用。

状态切换保护

```
ChangeState(newState)
→ 同状态跳过 (防重复)
→ Die 状态不可切出 (终态保护)
→ 更新 PreviousState / CurrentState → PlayStateAnimation
```

ForceChangeState 跳过同状态检查，专用于初始化场景 (Start 时强制进入 Idle)。

Clip 时长公开接口

```
float AttackClipLength => AttackClip != null ? AttackClip.length : 1f;
float HurtClipLength   => HurtClip   != null ? HurtClip.length   : 0.5f;
float DieClipLength    => DieClip    != null ? DieClip.length    : 1f;
```

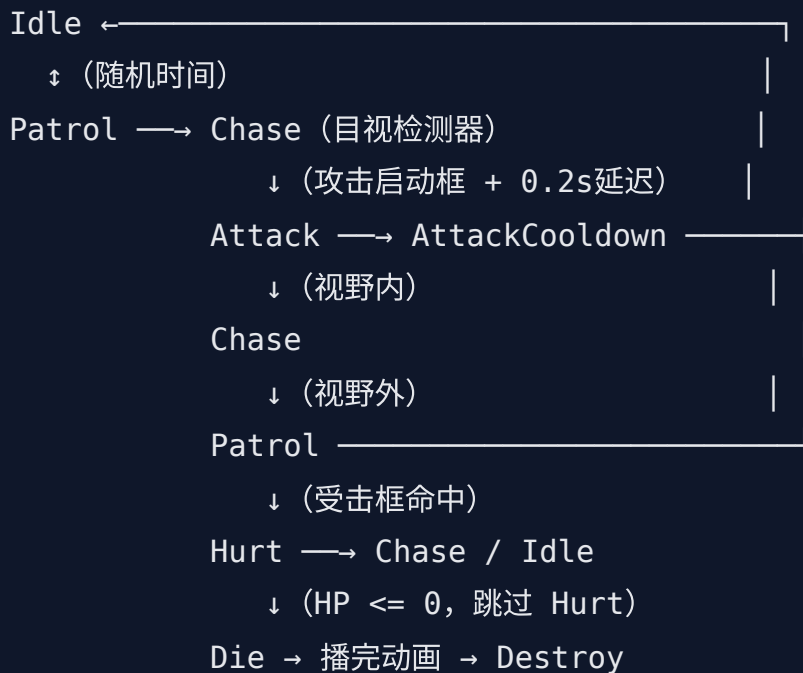
供 AI 控制器用作精确计时基准，避免硬编码时长魔法数字。

4.3 EnemyAI Melee (近战敌人 AI 逻辑层)

类型：MonoBehaviour，依赖 Rigidbody2D / EnemyStatBase / EnemyFSM

EnemyAI Melee 是近战敌人的行为大脑，纯逻辑层，不处理动画（委托给 EnemyFSM）。

状态流转图



转身检测器在玩家绕到背后时触发，延迟 0.7 秒转身，防止瞬间翻转的不真实感。

检测优先级 (Idle / Patrol 共用)

```
CheckDetectionAndReact()
1. 攻击启动框 (_attackTrigger.TargetInZone) → StartAttackDelay() → Attack
2. 目视检测器 (_visionDetector.TargetInZone) → Chase
3. 转身检测器 (_turnDetector.TargetInZone) → StartTurnAround() → 延迟转身
```

优先级从高到低，确保近距离时直接触发攻击，不会绕路进 Chase。

追逐：最后已知位置系统

```
UpdateChase()
→ 目视检测器有效 → 持续更新 _lastKnownPlayerPos，直接追玩家
→ 目视检测器失效 → _movingToLastKnown = true，前往最后已知位置
    → 距离 < 0.5f → SwitchToPatrol() (找不到玩家，放弃追逐)
```

模拟"失去视野后仍前往最后目击点搜索"的 AI 行为，增强敌人智能感。

攻击时序控制

```
UpdateAttack()
```

- stateTimer 从 AttackClipLength 倒计时
- stateTimer <= AttackClipLength * 0.5f (动画过半)
 - _hasActivatedHitbox = true → _attackHitbox.Activate()
- stateTimer <= 0 (动画播完)
 - 攻击框内有玩家 → SwitchToAttackCooldown()
 - 视野内无攻击框 → SwitchToChase()
 - 视野外 → SwitchToPatrol()

协程管理

```
StartAttackDelay() → AttackDelayCoroutine(): 等待 0.2s →
```

```
SwitchToAttack()
```

```
StartTurnAround() → TurnAroundCoroutine(): 等待 0.7s → Flip() → 判断下一状态
```

```
DieAfterAnimation() → 等待 DieClipLength → Destroy(gameObject)
```

```
CancelAllCoroutines(): 状态切换时统一取消所有协程, 防止协程与状态不同步
```

所有延迟行为均通过协程实现, 状态切换时 `CancelAllCoroutines()` 确保不留残留协程。

伤害处理

```
// 敌人攻击命中玩家 (AttackHitbox.OnHit 回调)
```

```
OnEnemyAttackHit(target)
```

- GetComponentInParent<PlayerStateMachine>()
- 计算伤害 (暴击判定) → playerSM.TakeHurt(damage)

```
// 玩家攻击命中敌人 (外部调用接口)
```

```
TakeDamage(damage)
```

- Die 状态中忽略
- _stat.CurHP -= damage
- HP <= 0 → SwitchToDie()
- HP > 0 → SwitchToHurt() (击退 + Hurt 状态)

五、战斗交互全链路

玩家攻击敌人的完整调用链：

```
玩家按 J 键
→ PlayerController:attackInput = true
→ HandlePlayerInput():stateMachine.TryAttack()
→ PlayerStateMachine:ChangeState(Attack1)
→ EnterState:controller.StartAttackMove(1f), PlayAnimation("Attack1")
→ HandleStateLogic:normalizedTime >= 0.5f → attackHitbox.Activate()
→ AttackHitbox:DetectionZone.OnTargetEnter 触发 → HandleTargetEnter
→ OnHit?.Invoke(target)
→ PlayerStateMachine.OnPlayerAttackHit(target)
→ enemyAI.TakeDamage(damage)
→ EnemyAIMelee:SwitchToHurt() → EnemyFSM.ChangeState(Hurt)
```

敌人攻击玩家的完整调用链：

```
EnemyAIMelee:UpdateAttack() → attackHitbox.Activate()
→ AttackHitbox:OnHit?.Invoke(target)
→ OnEnemyAttackHit(target)
→ playerSM.TakeHurt(damage)
→ PlayerStateMachine:stat.TakeDamage(damage)
→ HP <= 0 → ChangeState(Die)
→ HP > 0 → ChangeState(Hurt)
```

六、设计模式总结

模式	应用位置	作用
有限状态机 (FSM)	PlayerStateMachine / EnemyFSM	状态行为集中管理，避免条件嵌套膨胀
观察者模式	AttackHitbox.OnHit 事件	攻击判定与伤害逻辑解耦，双向战斗统一接口

策略模式	BlessingType 伤害公式	祝福效果以 switch 动态叠加，新增祝福只加分支
模板方法模式	EnemyStatBase	基类定义 Awake 初始化框架，子类 override 扩展
数据与逻辑分离	PlayerStat / EnemyStatBase	属性纯数据层，状态机不操作数值
组件复用模式	DetectionZone / AttackHitbox	一套通用检测组件服务玩家和敌人双方
懒加载模式	EnemyFSM.EnsureClipsApplied	动画资源首次播放时初始化，避免 Awake 顺序问题

感谢您的浏览！